

StatefulSet e DaemonSet

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare quando usare un **StatefulSet** invece di un **Deployment** e quali garanzie offre sull'identità dei Pod
- Configurare un **StatefulSet** con headless Service e **volumeClaimTemplates** per workload con storage persistente
- Descrivere **podManagementPolicy** e le differenze tra **OrderedReady** e **Parallel**
- Spiegare lo scopo di un **DaemonSet** e configurarlo per eseguire su nodi specifici tramite **nodeSelector** e **tolerations**
- Scegliere la strategia di aggiornamento corretta per un DaemonSet (**RollingUpdate** vs **OnDelete**)
- Distinguere i casi d'uso di **Deployment**, **StatefulSet** e **DaemonSet**

Teoria e Concetti Chiave

StatefulSet: Identità Stabile dei Pod

Un **StatefulSet** è un controller Kubernetes progettato per applicazioni che richiedono:

1. **Identità di rete stabile:** ogni Pod ha un hostname prevedibile e stabile (es. **mysql-0**, **mysql-1**, **mysql-2**)
2. **Storage persistente per Pod:** ogni Pod ha il suo Persistent Volume Claim dedicato che non viene eliminato al restart
3. **Ordinamento garantito:** il deploy, lo scaling e l'aggiornamento seguono un ordine preciso

 **Suggerimento:** Un **Deployment** assegna nomi casuali ai Pod (es. **webapp-6d9f7b8c9-xk21p**). Un **StatefulSet** assegna nomi con indice ordinale stabile (es. **mysql-0**, **mysql-1**). Questo è fondamentale per database e sistemi di messaggistica che si riferiscono ai peer per hostname.

Headless Service

Un **StatefulSet** richiede un **headless Service** (con **clusterIP: None**) che permette la risoluzione DNS diretta verso ogni singolo Pod:

```
# DNS generato automaticamente per ogni Pod del StatefulSet
<pod-name>.<headless-service-name>.<namespace>.svc.cluster.local

# Esempio per mysql-0 nel namespace myproject:
mysql-0.mysql-headless.myproject.svc.cluster.local
```

Questo permette ai peer di un cluster distribuito (es. MySQL Group Replication) di comunicare direttamente tra loro tramite hostname stabili.

volumeClaimTemplates

A differenza di un **Deployment** dove tutti i Pod condividono o montano gli stessi volumi, ogni Pod di uno **StatefulSet** ottiene il suo PVC dedicato tramite **volumeClaimTemplates**:

```
StatefulSet: mysql (replicas: 3)
├── Pod: mysql-0 → PVC: data-mysql-0 → PV: azure-disk-001
├── Pod: mysql-1 → PVC: data-mysql-1 → PV: azure-disk-002
└── Pod: mysql-2 → PVC: data-mysql-2 → PV: azure-disk-003
```

⚠ Attenzione: Se uno **StatefulSet** viene eliminato, i PVC **non vengono eliminati automaticamente** (a meno che non si usi **persistentVolumeClaimRetentionPolicy**). Questo è intenzionale per proteggere i dati.

podManagementPolicy

Policy	Comportamento
OrderedReady (default)	I Pod vengono creati/eliminati in ordine (0, 1, 2...). Il successivo parte solo quando il precedente è Ready
Parallel	I Pod vengono creati/eliminati tutti contemporaneamente (più veloce, ma perdi le garanzie di ordine)

Usare **Parallel** solo per StatefulSet dove l'ordine di avvio non è rilevante (es. semplici cache stateful).

Casi d'Uso di StatefulSet

Applicazione	Motivo per StatefulSet
Database relazionali (MySQL, PostgreSQL)	Storage persistente per Pod, hostname stabile per la replica
Database distribuiti (Cassandra, MongoDB)	Ogni nodo ha identità di rete stabile; peer discovery tramite DNS
Sistemi di messaggistica (Kafka, RabbitMQ)	Broker identificabili per hostname; storage per topic/queue
Cache persistente (Redis Cluster)	Cluster nodes si riferenziano per hostname; storage per RDB/AOF
Elasticsearch	Cluster nodes con identità stabile per master election

DaemonSet: Un Pod per Nodo

Un **DaemonSet** garantisce che **un'istanza (e una sola) di un Pod venga eseguita su ogni nodo** del cluster (o su un sottoinsieme di nodi). Quando un nuovo nodo viene aggiunto al cluster, il DaemonSet vi schedula automaticamente un Pod. Quando un nodo viene rimosso, il Pod viene eliminato.

Casi d'Uso di DaemonSet

Caso d'uso	Esempio
Raccolta di log dal nodo	Fluentd, Filebeat — raccolgono log da <code>/var/log</code> del nodo
Monitoraggio del nodo	Prometheus Node Exporter, Datadog Agent
Plugin di rete (CNI)	OVN-Kubernetes DaemonSet — gestisce il networking tra Pod
Storage distribuito	Ceph rook-ceph per storage distribuito su ogni nodo
Sicurezza	Falco, Sysdig per syscall monitoring su ogni nodo

nodeSelector e Tolerations

Per limitare il DaemonSet a un sottoinsieme di nodi si usano due meccanismi complementari:

nodeSelector — seleziona solo i nodi con determinate label:

```
nodeSelector:
  node-role.kubernetes.io/worker: ""
```

tolerations — permette di schedulare su nodi con **taint** (normalmente i taint "respingono" i Pod):

```
tolerations:
- key: node-role.kubernetes.io/infra
  operator: Exists
  effect: NoSchedule
```

 **Suggerimento:** Su ARO, i nodi infra e master hanno taint che impediscono lo scheduling di workload utente. Usa **tolerations** nei DaemonSet di sistema che devono girare anche su quei nodi.

Strategia di Aggiornamento del DaemonSet

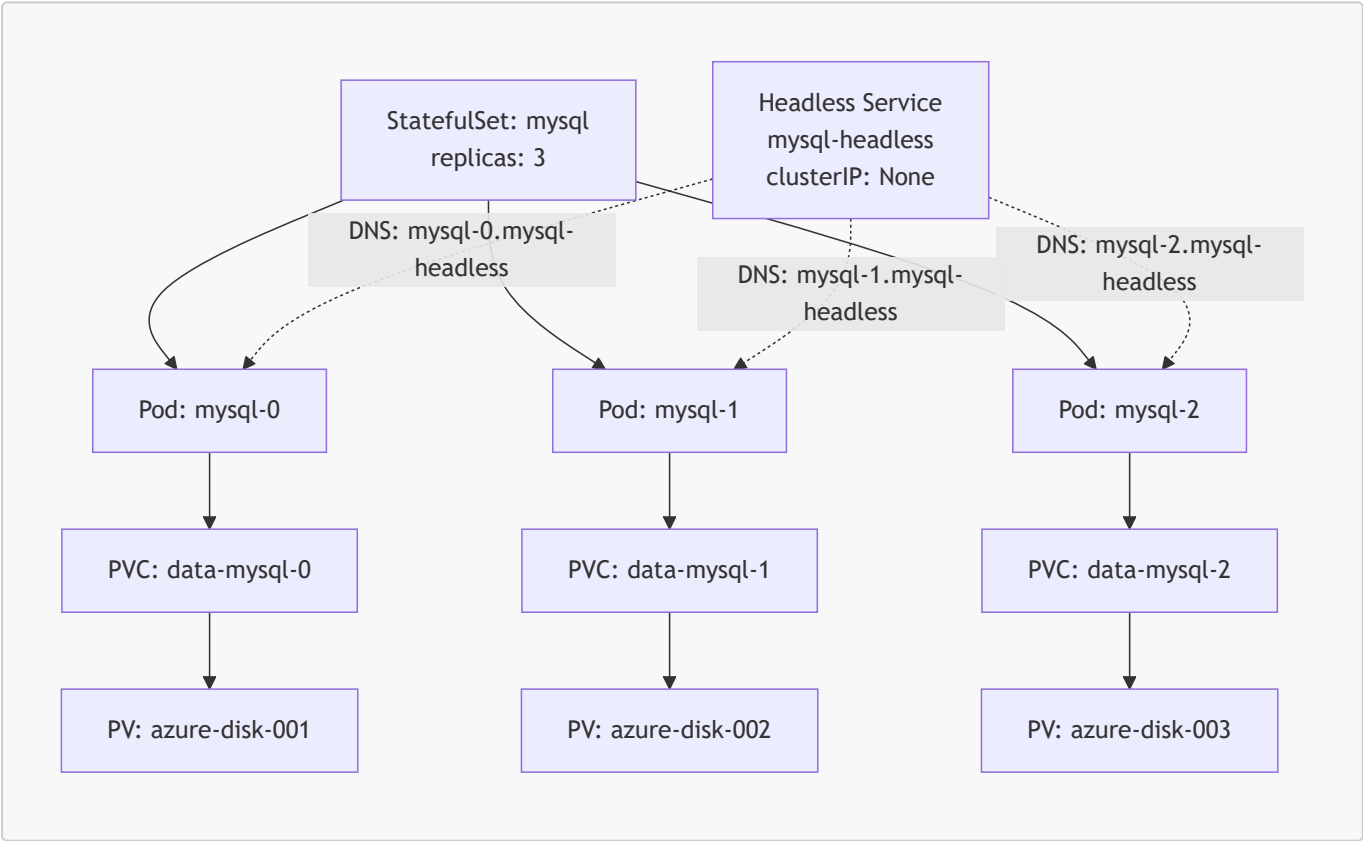
Strategia	Comportamento
RollingUpdate (default)	Aggiorna i Pod del DaemonSet uno alla volta (o in batch con maxUnavailable)
OnDelete	Il Pod viene aggiornato solo quando eliminato manualmente: utile per aggiornamenti controllati di agent critici

Tabella Comparativa: Deployment vs StatefulSet vs DaemonSet

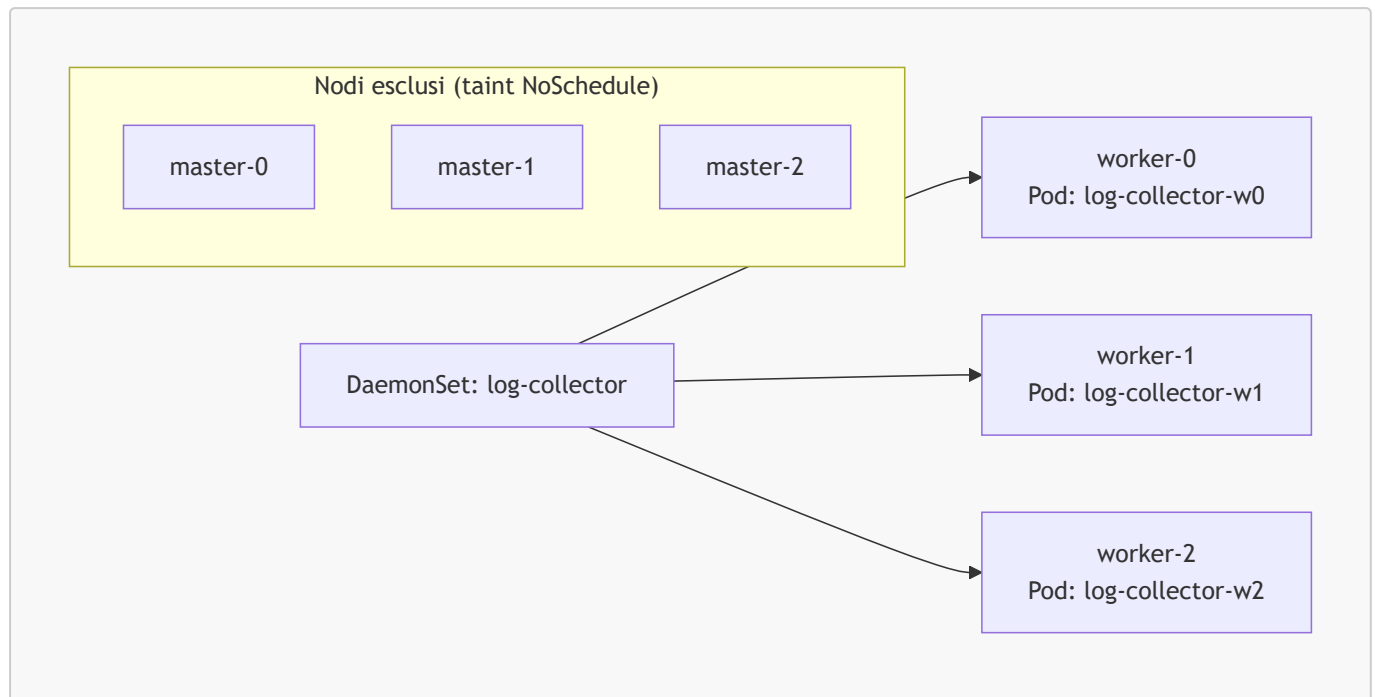
Caratteristica	Deployment	StatefulSet	DaemonSet
Nomi Pod	Casuali (app-xxxx- yyyy)	Ordinali stabili (app-0, app-1)	Legati al nodo (app- nodename)
Storage per Pod	Condiviso o nessuno	PVC dedicato per Pod	Tipicamente hostPath
Ordine di avvio	Nessun ordine	Ordinato (OrderedReady) o parallelo	Simultaneo su tutti i nodi
Scaling	Libero	Ordinato	N° Pod = N° nodi target
Caso d'uso	Microservizi stateless	Database, cluster distribuiti	Agent di sistema, monitoring, CNI

Diagrammi

StatefulSet con Headless Service e PVC dedicati



DaemonSet su tutti i Nodi Worker



Comandi Pratici con Output Atteso

```
# Elencare i StatefulSet nel progetto corrente
oc get statefulsets
```

```
# Output atteso:
```

```
# NAME      READY   AGE
# mysql     3/3     45m
# redis     2/2     12m
```

```
# Mostrare i dettagli di uno StatefulSet
oc describe statefulset mysql
```

```
# Output atteso (estratto):
```

```
# Name:      mysql
# Namespace: myproject
# Selector:   app=mysql
# Replicas:   3 desired | 3 total
# Update Strategy: RollingUpdate
# Pods Status: 3 Running / 0 Waiting / 0 Succeeded / 0 Failed
# Volume Claims:
#   Name:      data
#   StorageClass: managed-csi
#   Capacity:   10Gi
#   Access Modes: [ReadWriteOnce]
```

```
# Elencare i Pod di uno StatefulSet (nomi ordinali stabili)
oc get pods -l app=mysql
```

```
# Output atteso:
```

```
# NAME      READY   STATUS    RESTARTS   AGE
# mysql-0   1/1     Running   0           46m
# mysql-1   1/1     Running   0           44m
# mysql-2   1/1     Running   0           42m
```

```
# Elencare i PVC creati dallo StatefulSet
oc get pvc -l app=mysql
```

```
# Output atteso:
```

```
# NAME              STATUS    VOLUME             CAPACITY   ACCESS MODES   STORAGECLASS  AGE
# data-mysql-0      Bound    pvc-abc123...      10Gi       RWO             managed-csi   46m
# data-mysql-1      Bound    pvc-def456...      10Gi       RWO             managed-csi   44m
# data-mysql-2      Bound    pvc-ghi789...      10Gi       RWO             managed-csi   42m
```

```
# Scalare uno StatefulSet (scaling ordinato: parte/ferma nell'ordine 0, 1, 2...)
oc scale statefulset mysql --replicas=5
```

```
# Output atteso:
```

```
# statefulset.apps/mysql scaled
```

```
# Elencare i DaemonSet nel progetto corrente
oc get daemonsets
```

```
# Output atteso:
```

```
# NAME              DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE
SELECTOR    AGE
# log-collector     3         3         3       3             3           <none>
2h
```

```
# Mostrare i dettagli di un DaemonSet e i suoi nodi target
oc describe daemonset log-collector
```

```
# Output atteso (estratto):
```

```
# Name:              log-collector
# Selector:          app=log-collector
```

```
# Node-Selector:  node-role.kubernetes.io/worker=
# Desired Number of Nodes Scheduled: 3
# Current Number of Nodes Scheduled: 3
# Number of Nodes Scheduled with Up-to-date Pods: 3
```

```
# Forzare l'aggiornamento di un DaemonSet con strategia OnDelete
# (elimina il Pod su un nodo specifico: il DaemonSet ne creerà uno aggiornato)
oc delete pod log-collector-w0

# Output atteso:
# pod "log-collector-w0" deleted
# (il DaemonSet ricrea automaticamente il Pod con la nuova immagine)
```

```
# Verificare su quali nodi stanno girando i Pod di un DaemonSet
oc get pods -l app=log-collector -o wide

# Output atteso:
```

# NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
# log-collector-6xk2p	1/1	Running	0	2h	10.128.2.10	worker-0
# log-collector-mn9qr	1/1	Running	0	2h	10.128.3.11	worker-1
# log-collector-pz7ts	1/1	Running	0	2h	10.129.1.12	worker-2

Esempi YAML Completi

StatefulSet con headless Service e volumeClaimTemplates

```
# Headless Service obbligatorio per lo StatefulSet
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
  namespace: myproject
  labels:
    app: mysql
spec:
  # clusterIP: None → headless: nessun IP virtuale, DNS risolve direttamente ai Pod
  clusterIP: None
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
      name: mysql
  ---
apiVersion: apps/v1
```

```
kind: StatefulSet
metadata:
  name: mysql
  namespace: myproject
spec:
  # Nome del headless Service che fornisce l'identità di rete
  serviceName: "mysql-headless"
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  # Ordine garantito: mysql-1 parte solo dopo che mysql-0 è Ready
  podManagementPolicy: OrderedReady
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      # Aggiorna al massimo 1 Pod alla volta
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: registry.redhat.io/rhel9/mysql-80:latest
          ports:
            - containerPort: 3306
              name: mysql
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
              memory: "1Gi"
              cpu: "1000m"
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: root-password
            - name: MYSQL_DATABASE
              value: "appdb"
          # Monta il volume dal PVC dedicato al Pod
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
          livenessProbe:
            tcpSocket:
              port: 3306
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
```



```

      exec:
        command: ["mysqladmin", "ping", "-h", "127.0.0.1"]
        initialDelaySeconds: 10
        periodSeconds: 5
# Template per la creazione automatica di PVC per ogni Pod
volumeClaimTemplates:
- metadata:
    name: data
  spec:
    accessModes: ["ReadWriteOnce"]
    # Su ARO: managed-csi usa Azure Managed Disk (Premium SSD default)
    storageClassName: managed-csi
    resources:
      requests:
        storage: 10Gi

```

DaemonSet con nodeSelector e tolerations

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
  namespace: myproject
  labels:
    app: log-collector
spec:
  selector:
    matchLabels:
      app: log-collector
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      # Aggiorna al massimo 1 nodo alla volta
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: log-collector
    spec:
      # Esegui solo sui nodi con il ruolo 'worker'
      nodeSelector:
        node-role.kubernetes.io/worker: ""
      # Permetti lo scheduling anche su nodi con taint specifici (se necessario)
      tolerations:
        - key: node-role.kubernetes.io/infra
          operator: Exists
          effect: NoSchedule
      containers:
        - name: fluentd
          image: registry.redhat.io/openshift-logging/fluentd-rhel9:latest
          resources:

```

```
    requests:
      memory: "200Mi"
      cpu: "100m"
    limits:
      memory: "500Mi"
      cpu: "200m"
  # Monta i log del nodo host (tipico pattern DaemonSet per log collector)
  volumeMounts:
    - name: varlog
      mountPath: /var/log
      readOnly: true
    - name: dockercontainers
      mountPath: /var/lib/docker/containers
      readOnly: true
  terminationGracePeriodSeconds: 30
  volumes:
    # Log del sistema operativo del nodo host
    - name: varlog
      hostPath:
        path: /var/log
    # Log dei container (directory Docker/CRI-O sul nodo)
    - name: dockercontainers
      hostPath:
        path: /var/lib/docker/containers
```

Note Specifiche per Azure ARO

StorageClass per StatefulSet su ARO

ARO fornisce diverse StorageClass per i PVC degli StatefulSet:

StorageClass	Tipo Azure	Access Mode	Caso d'uso
<code>managed-csi</code>	Azure Managed Disk (Standard HDD)	<code>ReadWriteOnce</code>	DB dev/test
<code>managed-premium</code>	Azure Managed Disk (Premium SSD)	<code>ReadWriteOnce</code>	DB produzione
<code>azurefile-csi</code>	Azure Files (SMB)	<code>ReadWriteMany</code>	Condivisione tra Pod
<code>azurefile-premium</code>	Azure Files Premium	<code>ReadWriteMany</code>	Condivisione ad alte prestazioni

```
# Verificare le StorageClass disponibili su ARO
oc get storageclass

# Output atteso:
# NAME                                PROVISIONER                                RECLAIMPOLICY
VOLUMEBINDINGMODE
# managed-csi                        disk.csi.azure.com                        Delete
WaitForFirstConsumer
# managed-premium                    disk.csi.azure.com                        Delete
WaitForFirstConsumer
# azurefile-csi                      file.csi.azure.com                        Delete          Immediate
# azurefile-premium                  file.csi.azure.com                        Delete          Immediate
```

⚠ Attenzione: Azure Managed Disk (`ReadWriteOnce`) non può essere montato da più Pod contemporaneamente. Per StatefulSet che necessitano di storage condiviso tra Pod usare `azurefile-csi` (`ReadWriteMany`).

DaemonSet e Nodi Infra su ARO

Su ARO esistono nodi infra gestiti da Red Hat/Microsoft (router, registry, monitoring) con taint che impediscono lo scheduling di workload utente. Se un DaemonSet di logging o monitoring deve girare anche su questi nodi, aggiungi esplicitamente le toleration:

```
tolerations:
- key: node-role.kubernetes.io/infra
  operator: Exists
  effect: NoSchedule
- key: node.kubernetes.io/not-ready
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 300
```

⚠ Attenzione: I nodi master su ARO sono completamente gestiti da Red Hat/Microsoft e non è possibile schedulare DaemonSet su di essi, indipendentemente dalle toleration configurate.

SCC per StatefulSet su ARO

I workload stateful come database spesso necessitano di scrivere su storage con permessi specifici. Se un container richiede di girare come utente specifico:

```
# Verificare quale SCC è necessaria per un StatefulSet
oc adm policy scc-subject-review -f statefulset.yaml

# Concedere una SCC più permissiva al ServiceAccount del StatefulSet (solo se necessario)
oc adm policy add-scc-to-serviceaccount anyuid -z mysql-sa -n myproject
```

Riepilogo dei Punti Chiave

- **StatefulSet** fornisce Pod con nomi ordinali stabili, storage persistente per Pod tramite **volumeClaimTemplates** e un headless Service per DNS diretto
 - I PVC creati da uno StatefulSet **non vengono eliminati** quando il StatefulSet viene eliminato: i dati sono protetti
 - **podManagementPolicy**: **OrderedReady** garantisce che i Pod partano/fermino in ordine; **Parallel** li gestisce tutti contemporaneamente
 - Un **DaemonSet** assicura che un Pod giri su ogni nodo (o sottoinsieme di nodi): utile per log collector, monitoring agent, CNI plugin
 - **nodeSelector** limita il DaemonSet a nodi con label specifiche; **tolerations** permette lo scheduling su nodi con taint
 - La strategia **RollingUpdate** aggiorna i Pod del DaemonSet progressivamente; **OnDelete** solo quando eliminati manualmente
 - Su ARO, usare **managed-premium** per database in produzione e verificare le SCC per workload con requisiti di utenza speciali
-

Domande di Verifica

1. Qual è il principale vantaggio dell'uso di `volumeClaimTemplates` in uno `StatefulSet` rispetto ai volumi di un `Deployment`?

- A) I PVC degli `StatefulSet` sono più veloci di quelli usati nei `Deployment`
- B) Ogni Pod ottiene il proprio PVC dedicato con dati persistenti e indipendenti dagli altri Pod ☒
- C) I `volumeClaimTemplates` permettono di usare `ReadWriteMany`
- D) I PVC degli `StatefulSet` vengono eliminati automaticamente insieme al `StatefulSet`

2. Cosa rende "headless" un `Service` con `clusterIP: None`?

- A) Il `Service` non espone traffico verso i Pod
- B) Il `Service` non ha un IP virtuale: il DNS risolve direttamente agli IP dei singoli Pod ☒
- C) Il `Service` funziona solo all'interno del namespace
- D) Il `Service` non supporta il bilanciamento del carico

3. Un `DaemonSet` deve girare su tutti i nodi worker ma non sui nodi infra. Quale campo va configurato?

- A) `spec.selector.matchLabels`
- B) `spec.template.spec.tolerations`
- C) `spec.template.spec.nodeSelector` ☒
- D) `spec.updateStrategy`

4. Con `podManagementPolicy: OrderedReady` e 3 repliche, in quale ordine vengono creati i Pod?

- A) In ordine casuale
- B) Tutti e 3 simultaneamente
- C) `mysql-2` → `mysql-1` → `mysql-0` (ordine inverso)
- D) `mysql-0` → `mysql-1` → `mysql-2`, ognuno attende che il precedente sia Ready ☒

5. Su ARO, quale `StorageClass` è consigliata per un database MySQL in produzione con singolo Pod?

- A) `azurefile-csi` (Azure Files)
- B) `managed-premium` (Azure Disk Premium SSD, `ReadWriteOnce`) ☒
- C) `azurefile-premium` (Azure Files Premium)
- D) `managed-csi` (Azure Disk Standard HDD)